

Namaste! Welcome to the class. As an AI Engineer, I can tell you that learning LangChain will completely change how you approach software architecture for Generative AI.

Agar aap backend development ke layered architecture ko samjhte hain—jaise controllers, services, aur repositories ek ke baad ek connected hote hain—toh LangChain आपको बहुत familiar lagega. Yeh essentially LLMs ke liye wahi framework aur structure provide karta hai. Let's dive into your first masterclass covering the absolute basics of LangChain and simple Chains, strictly following our 15-step structure.

Topic: LangChain Fundamentals & Creating Your First Chain

1. Real-world analogy first

Imagine karo ek बहुत hi badhiya raw LLM (jaise GPT-4 ya Gemini) ek world-class *Chef* hai. Agar aap directly API use karte ho, toh आपको khud us Chef ko raw materials (prompt) देने padenge, uski complicated language samajhni padegi, aur final dish (output) khud garnish karni padegi.

LangChain ek pura Restaurant Management System hai. Isme waiters (Prompt Templates), kitchen managers (Chains), assistants (Agents), aur recipe books (Memory) hain. Ye sab milkar us "raw Chef" ko ek end-to-end production-ready system banate hain.

2. Why this concept is needed (Why LangChain was created?)

Jab hum GenAI applications banate hain, sirf ek model ko text bhej kar reply lena kaafi nahi hota. Real-world apps mein humein LLM ko apne database se connect karna hota hai, purani chat history yaad rakhni hoti hai, ya internet search allow karna hota hai. Raw APIs yeh sab naturally support nahi karti. LangChain in sab disconnected components ko ek standard interface ke through jodne (chaining) ke liye banaya gaya tha.

3. Problem it solves (Problems faced while using raw LLM APIs)

Raw API endpoints use karne mein kuch major bottlenecks aate hain:

- **Prompt Spaghetti:** Har task ke liye manually lambe strings concatenate karna code ko messy bana deta hai.
- **Statelessness:** Raw LLM API stateless hoti hai. Har request fresh hoti hai. Context manage karna बहुत painful ho jata hai.
- **Vendor Lock-in:** Agar aaj aap OpenAI use kar rahe ho aur kal आपको Gemini ya Llama 3 par switch karna ho, toh raw APIs ke format alag hone ki wajah se poora code rewrite karna padta hai. LangChain is lock-in ko khatam karta hai.

4. Internal working (What happens behind the scenes & What are Chains?)

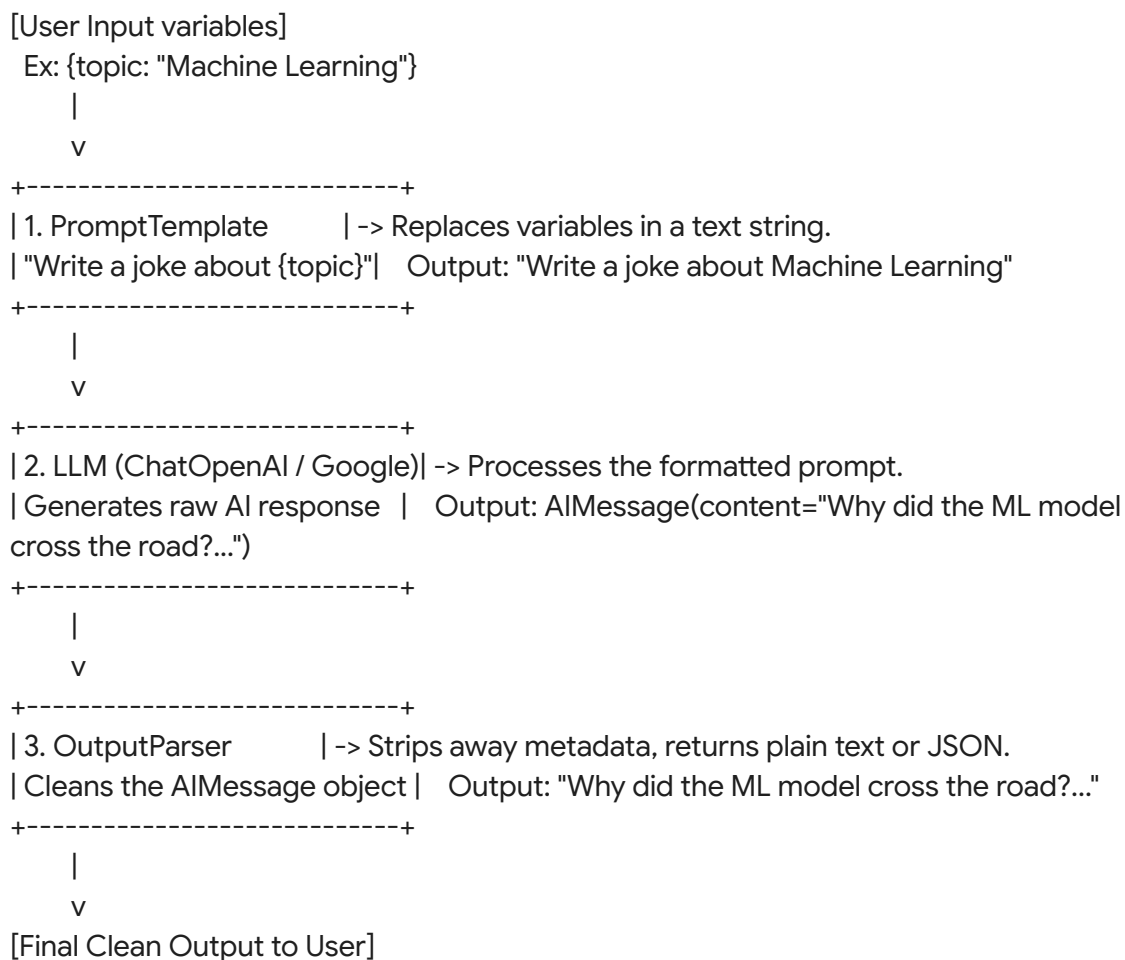
Background mein LangChain koi naya AI magic nahi karta, balki yeh ek **orchestration layer** hai. Iska sabse core concept hai "**Chain**". Jaise data structures mein Linked List ke nodes ek dusre se strictly sequential order mein connected hote hain (ek node ka data next node ka input banta hai), waise hi LangChain mein components chain hote hain. Ek Simple Chain mein data ka flow kuch is tarah hota hai: aap variables dete ho, ek template unko string (prompt) mein badalta hai, wo string LLM ke paas jati hai, aur LLM ka response ek parser clean format mein

aapko return karta hai.

Modern LangChain mein isko **LCEL (LangChain Expression Language)** kehte hain, jo Unix/Linux ke pipe | operator ka use karke components ko beautifully connect karta hai.

5. Architecture/flow explanation using text diagrams

A simple LangChain architecture flow looks like this:



6. Step-by-step explanation

- **Step 1:** Apne Python environment mein dependencies install karein: `pip install langchain langchain-openai python-dotenv`
- **Step 2:** Secure API key management ke liye ek `.env` file banayein aur load karein.

- **Step 3:** LLM class ka object initialize karein (e.g., ChatOpenAI).
- **Step 4:** Ek PromptTemplate banayein jisme hum {variables} define karenge.
- **Step 5:** LCEL piping (|) ka use karke apne chain ko jodein.
- **Step 6:** Chain ko apne data ke sath invoke karein.

7. Complete Python code with comments

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Load Environment Variables (API Keys)
load_dotenv()

# 2. Initialize the LLM (Using standard OpenAI wrapper here)
# temperature=0.7 means a bit creative, 0 means strict/factual
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# 3. Create a Prompt Template
# We use {company_product} as a placeholder variable
template = "You are a branding expert. Suggest a catchy company name for a startup that makes {company_product}."
prompt_template = PromptTemplate.from_template(template)

# 4. Initialize an Output Parser (Converts AI message object to raw string)
parser = StrOutputParser()

# 5. Build the Chain using LCEL (LangChain Expression Language)
# The pipe '|' operator passes the output of the left side as input to the right side
chain = prompt_template | llm | parser

# 6. Execute (Invoke) the chain
if __name__ == "__main__":
    product_type = "AI-powered medical devices"
    print("Generating name...")
```

```
# We pass the variable expected by the prompt template
result = chain.invoke({"company_product": product_type})

print(f"\nFinal Result:\n{result}")
```

8. Dry run of the code line by line

1. `load_dotenv()`: Hamare `.env` file se `OPENAI_API_KEY` ko environment variables mein load karta hai.
2. `llm = ChatOpenAI(...)`: Ye LLM connection object banata hai. Model aur strictness (temperature) set karta hai.
3. `template = "..."`: Hamara raw prompt jisme `{company_product}` ki jagah khali chhodi gayi hai.
4. `parser = StrOutputParser()`: Ye ensure karega ki humein end mein sirf text mile, koi extra API metadata nahi.
5. `chain = prompt_template | llm | parser`: **The Magic Line.** Yahan hum pipeline define kar rahe hain. "Pehle prompt template mein data dalo, jo prompt banega usko LLM ko do, LLM ka jo answer aayega usko parser se clean karo."
6. `chain.invoke({"company_product": "..."})`: Yahan actually chain execution start hota hai aur data flow karta hai.

9. Input and output examples

Input: `{"company_product": "smart shoes for blind people"}`

Output: "NaviStep Technologies"

10. Common mistakes beginners make

- **Hardcoding API Keys:** Beginner direct code mein API key likh dete hain aur GitHub par push kar dete hain. Hamesha `python-dotenv` use karein.
- **Forgetting Output Parser:** Agar aap parser nahi lagayenge, toh return mein ek `AIMessage` object aayega jisme bohot saara useless metadata (token count, logprobs) hoga, jisse aapka backend crash ho sakta hai agar wo string expect kar raha tha.
- **Typo in Prompt Variables:** Template mein `{topic}` likha aur invoke karte waqt `chain.invoke({"subject": "math"})` bhej diya. Code fat jayega.

11. Interview questions and answers

Q: What is LCEL and why did LangChain introduce it over the old LLMChain?

Answer: LCEL stands for LangChain Expression Language. It is a declarative way to compose chains using the `|` operator. It was introduced to provide out-of-the-box support for streaming, asynchronous execution, and parallel processing, which were very hard to implement with the legacy `LLMChain` class.

Q: Can we switch the LLM provider easily in LangChain?

Answer: Yes. Because of LangChain's abstraction layer, changing from OpenAI to Gemini is as simple as replacing `ChatOpenAI()` with `ChatGoogleGenerativeAI()`. The rest of the chain logic

remains exactly the same.

12. Best practices used in industry

- **Modularization:** Prompts ko code file se alag ek separate YAML ya text file mein rakhein taaki non-tech (Product Managers) bhi unko tweak kar sakein bina Python code tode.
- **Use .invoke() vs .stream():** Chhote tasks ke liye .invoke() theek hai, lekin user-facing UI ke liye hamesha .stream() use karein taaki user ko typing effect dikhe aur app fast feel ho.

13. Small practice exercises

- **Task 1:** Change the PromptTemplate to act as a "Code Reviewer" and pass a small Python function string as a variable to get a review.
- **Task 2:** Try removing the StrOutputParser() from the chain and print the output. Notice the difference in the object structure.

14. Mini assignment

Build an "Email Generator App".

Create a chain that takes two variables: {sender_name} and {purpose}. The prompt should generate a professional email. Test it with inputs like sender="Gagan" and purpose="Asking for a 2-day leave due to a family trip".

15. Summary notes for revision

- LangChain is an orchestration framework, not an AI model itself.
- It solves Prompt Management, Memory, and Tool integration.
- **Chain Concept:** Connecting components sequentially (Prompt -> LLM -> Parser).
- **LCEL:** The modern way to write chains using the pipe | operator.
- Always use StrOutputParser to clean up the raw LLM response.

That wraps up the core fundamentals! Are you comfortable with how LCEL uses the pipe operator here, or should we build a slightly more complex chain next to see how we can pass the output of one LLM directly into another as a sequential chain?